

MOAR Console — visual walkthrough

A flip-through of each console view, captured live from the marimo app, with the intent of each view annotated underneath. Generated from console [@e2be954](#).

Walkthrough

Startup > { Strategy > [Pick components, Vault & Matrix], Configuration }

Flow > [Land, Health, Migrate]

Analyze

These are real screenshots of the live UI, each shown in full — the marimo scroll shell is released at capture time so below-fold panels are not cropped. The data-driven panels (Health, Analyze) show their honest pre-audit / no-data states because the demo stack is not running: the gate never bluffs a pass, so an un-run audit reads as unmeasured, not green. Regenerate this PDF with **bash flipthrough/build.sh** after a console change.



MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

- Walkthrough
- Startup
- Flow
- Analyze

Guided walkthrough

The golden path — the operator workflow, end to end.

1•Setup → 2•Config → 3•Land → 4•Gate → 5•Analyze → 6•Migrate

1 of 6 value moments demonstrated live in this session; the rest light up as you walk the tabs. A step is live only when a real signal backs it — an undemonstrated step reads waiting, never a bluffed green.

1. Declare your constraints, get a fitting open stack — • ready · spec saved — the fitting stack is locked in

▼
2. Wire a source and watch the OCSF mapping applied — • ready · previewing the raw → OCSF mapping for ocsf

▼
3. Deploy and watch data land per OCSF class — ◦ waiting · deploy and run the pipeline to watch data land per class

▼
4. The data-health gate certifies it — and every engine agrees — ◦ waiting · gate amber — unproven layers remain; run them in Flow › Health

▼
5. Run a real hunt over the landed OCSF and get a finding — ◦ waiting · run a hunt over the landed table to surface a finding

▼
6. Swap any component — being wrong is cheap — ✓ live · a swap was proven not to change the answer (cross-engine equality)

▼

Intent. The guided walkthrough — the golden path (the operator workflow end to end) threaded across the tabs as a progressive-disclosure stepper: declare constraints → wire a source and watch the OCSF mapping → land data per OCSF class → the data-health gate certifies it and every engine agrees → a hunt fires over the landed OCSF → any component is reversible. Each step names where it lives and shows a live status — 'live' only when a real signal backs it, 'waiting' until the value moment is actually demonstrated, never a bluffed green. It is the product's narrative spine; the flip-through you're reading is the standing visual record of it.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Walkthrough

Startup

Flow

Analyze

Strategy

Configuration

Pick components

Vault & Matrix

Reference architectures — a known-good starting point

Validated patterns from the book (Appendix C). Match one, then tune; the constraint and anti-pattern checks still apply to whatever you land on.

Lean single-engine lakehouse

A small team (2-3 engineers), cost-led, mixed analyst workloads with no heavy concurrency. One engine over shared Iceberg covers most of what a SOC asks before a second engine earns its keep.

Storage seaweedfs · Catalog polaris · Ingest vector · Query datafusion · Schema ocsf

Cost: Lowest: warm Iceberg-on-S3 storage floor, one engine to run. Appendix C; Ch5 (single-engine over shared Iceberg).

Workload-optimized multi-engine

Mixed real-time detection and threat hunting with 3-5 engineers: a fast aggregation engine (ClickHouse) beside the Iceberg-native default (DataFusion), both over the same tables.

Storage seaweedfs · Catalog polaris · Ingest vector · Query datafusion, clickhouse · Schema ocsf

Cost: Warm storage floor plus a second engine's compute and on-call. Add the second engine only for a workload the first can't serve. Appendix C; Ch3 WorkLoad 1+2.

Air-gapped on-prem

An on-prem or data-sovereignty mandate that rules out cloud-managed services. Self-hosted object store + REST catalog + a federating MPP engine, all inside the boundary.

Storage dell_ecs · Catalog polaris · Ingest nifi · Query trino · Schema ocsf

Cost: Appliance + self-hosted compute; no cloud egress, but you own the ops. Appendix C; Ch3 §3.2 C3 (on-prem / sovereignty).

Cost-aggressive route-by-value

High-volume, low-value-heavy telemetry where the binding constraint is cost: route-by-value at ingest drops 70-90% of the volume before it lands, on flat no-egress storage.

Storage wasabi · Catalog polaris · Ingest cribl · Query datafusion · Schema ocsf

Cost: Lowest end to end: most volume never lands, and what does sits on the cheapest warm tier. Appendix C; Ch3 Workload 5 (route-by-value).

Constraints — declare these first

The binding constraint decides which components are even reachable, before capability decides which is best (Ch3). Set these; the picker below flags what each rules out.

Deployment

Cloud is fine

Team capacity

3-5 engineers

Vendor posture

Open-source first

Primary workload

Balanced / mixed

Compliance

Select...

Cost priority

Balanced

What your constraints rule on (current picks)

No binding constraint declared — every component is reachable. Declare a constraint to filter.

Reachable after constraints (best-fit pick): storage 5/5 → seaweedfs · catalog 6/6 → polaris · ingest 5/5 → vector · query 6/6 → datafusion · schema 6/6 → ocsf

No declared constraint touches the current selection.

Modular Component Selection

Choose the components for your active MOAr stack deployment.

Storage · C6

☒ SeaweedFS

☐ MinIO

☐ AWS S3

☐ Wasabi

☐ Dell ECS

Catalog · C2

☒ Polaris

☐ Nessie

☐ Lakekeeper

☐ Hive Metastore (HMS)

☐ Unity Catalog OSS

☐ AWS Glue

Ingest · C4

☒ Vector

☐ Fluent Bit

☐ Apache NiFi

☐ Cribl Stream

☐ Tenzir

Query Engine(s) · C3

☒ DataFusion

☐ ClickHouse

☐ StarRocks

☐ Dremio

☐ DuckDB

☐ Trino

Schema Standard

☒ OCSF

☐ ECS

☐ Splunk CIM

☐ ASIM

☐ CEF

☐ Raw

These five pickers are the CapabilityMatrix components you choose among — C6 storage tier, C2 catalog, C4 ingestion/route, C3 query engine — plus the schema / normalization standard (OCSF and its alternatives), the data contract the Matrix treats as cross-cutting (C1 scores the table format, which the reference stack writes as Iceberg). The Matrix's other components aren't picks here: C0 Substrate pattern; C1 Lakehouse / table format; C5 Visualization, UX & detection rules; C7 Streaming pipeline; C8 Agentic / authoring / interface. The full scored C0–C8 model is the CapabilityMatrix.

Compatibility & Operational Notes

- Polaris / Lakekeeper + SeaweedFS — use static keys**

AWS STS credential vending (AssumeRole) isn't served by SeaweedFS, so set the warehouse to static S3 keys + a path-style endpoint (Polaris #3640/#3742, Lakekeeper #8312). Pin Polaris >= 1.4.1 (CVE-2026-42810).
- DataFusion + nested OCSF**

DataFusion reads nested structs/lists/maps natively. The one known edge is *additive schema evolution* on List: if a struct inside a list (e.g. observables[]) gains a new nullable field and you scan across mixed-version Parquet, planning can fail (apache/datafusion#20835, open, seen on 52.1.0). A single-schema OCSF table is unaffected — pin an explicit schema.

Design anti-patterns (Appendix B)

- Iceberg maintenance has no owner**

None of the selected query engines runs Iceberg table maintenance on its own. Without scheduled compaction, snapshot expiration, and orphan-file cleanup, small files accumulate and reads slow over time — budget a maintenance job (Spark or a pyiceberg cron) as part of the deploy (Appendix B: Iceberg maintenance at scale).

Cost-to-serve — storage floor for this volume

Raw ingest TB/day

1

Retention window

7 years

At 1 TB/day held 7-year, the storage floor for an Iceberg-on-S3 lakehouse is about 6,908/mo **, against about **102,009/mo for the same events on a hot search index — roughly 14.8x, a \$95,101/mo gap. A cold Glacier-IR tier widens it to about 97x (storage only, excludes retrieval). Storage floor, not a TCO model; ratios are corpus-specific.

Intent. The constraint-first decision flow. You declare the binding constraints first (deployment, team size, vendor posture, workload, compliance, cost), and the funnel narrows the full component catalog to a fit-justified shortlist — disqualify / caution / favor per candidate, with a reachable-N/M count and a best-fit pick. This is the public half of the Capability Matrix's method (constraint-conditional weighting) shown live and free; the scored ranking — which specific tool wins for your workload archetype and by how much — is the paid Matrix at /matrix.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Walkthrough

Startup

Flow

Analyze

Strategy

Configuration

Pick components

Vault & Matrix

Architecture Strategy & OKF Vault

This panel reads the project1 strategy vault as a **Google Open Knowledge Format (OKF v0.1)** bundle — a directory of markdown files whose type: frontmatter and `[[wikilinks]]` form a knowledge graph (Google Cloud, published 2026-06-12). Decisions (MDR-xxxx) and Assumptions (A-xx) are read straight off disk by `okf_reader`, so the stack you pick above stays coupled to the recorded *why*. Tolaria indexes this same vault for the agent host; this app reads the files directly rather than through Tolaria's read-only MCP server.

Search vault:

Search decisions & assu

Decision Records (MDRs) — 32 match:

MDR-0027

— Adopt augment-vs-replace decision-path scoring (Accepted, 2026-06-17) · MDR-0027-augment-vs-replace-decision-path-scoring.md

MDR-0010

— Pipelines (C4) criterion taxonomy — 9 criteria (Accepted, 2026-05-25) · MDR-0010-pipelines-criterion-taxonomy.md

MDR-0023

— Engines criterion addition — native_ip_type_support (Accepted, 2026-05-25) · MDR-0023-engines-native-ip-type.md

MDR-0015

— Default 6-month revalidation cadence, faster on benchmark triggers (Accepted, 2026-05-25) · MDR-0015-revalidation-cadence.md

MDR-0031

— Ratify the illustrative-worked-example exception to the public/paid line (Accepted, 2026-06-17) · MDR-0031-illustrative-worked-example-public-exception.md

MDR-0004

— Score per archetype with conditional weights, not absolute weights (Accepted, 2026-05-25) · MDR-0004-archetype-conditional-weights.md

MDR-0005

— Cap criteria per component at 3-14 dimensions (Accepted, 2026-05-25) · MDR-0005-dimensionality-3-to-14.md

MDR-0009

— Formats+Catalogs Archetype A weights (multi-engine open lakehouse) (Accepted, 2026-05-25) · MDR-0009-formats-catalogs-archetype-a-weights.md

MDR-0025

— F+C scoring — Nessie and Polaris share metadata-fetch latency at toyscale; no catalog-driver overhead differentiator on read path (Proposed, 2026-05-25) · MDR-0025-nessie-polaris-metadata-fetch-parity.md

MDR-0017

— Engines Archetype B weights (multi-source federated lakehouse) (Accepted, 2026-05-25) · MDR-0017-engines-archetype-b-weights.md

Strategic Assumptions — 16 match:

A-06

— Iceberg V3 features (row lineage, deletion vectors, variant type) are rolling across engines through 2026; engine support varies by version ... (confidence high, reviewed 2026-05-25)

A-15

— Vendor self-published benchmarks (StarRocks 5-30s multi-table claim, Cribl 70-90% reduction, etc.) are Tier C until independently corroborat... (confidence high, reviewed 2026-05-25)

A-05

— Unity Catalog self-hosted will remain operationally immature relative to Databricks-hosted through the next revalidation cycle (until 2026-1... (confidence high, reviewed 2026-05-25)

A-17

— A compliance requirement is binary (a regime either mandates it or it does not, so a candidate either clears the gate or is disqualified) an... (confidence medium, reviewed 2026-06-17)

A-12

— AWS SecurityLake hidden-cost delta (20-40% above advertised pricing) is structurally persistent for multi-cloud teams (confidence medium, reviewed 2026-05-25)

A-04

— Apache Polaris ecosystem adoption will accelerate through 2026 as Apache governance + Snowflake commercial momentum compound. Current thin-o-... (confidence medium, reviewed 2026-05-25)

A-13

— Trino federation latency floor (~2.67s on Zeek conn.log workload) is structural (coordinator + multi-source planning overhead), not tunable ... (confidence medium, reviewed 2026-05-26)

A-09

— A 6-month default revalidation cadence matches the median upstream-evidence change rate (engine releases, catalog GA timelines, table-format... (confidence medium, reviewed 2026-05-25)

A-01

— H3-PERFORMANCE-01 (ClickHouse 145x faster than Splunk on 10M Zeek conn.log) generalizes from single-node sequential workloads to distributed... (confidence medium, reviewed 2026-05-25)

A-10

— existing_in_house_skill_base is structurally client-specific and not scoreable at standing-matrix level (confidence high, reviewed 2026-05-25)

Selected components — pros, cons, reversibility & provenance

SeaweedFS

Pros:

Fast small-file lookups, lightweight, built-in volume replication; fits local S3 testing.

Cons:

AWS STS credential vending isn't served, so REST catalogs need static S3 keys + path-style.

Swap cost (reversibility):

Low — same S3 API as MinIO/AWS S3, so a swap is config + redeploy with no table rewrite. Moving the existing data is an S3 copy, not a re-land.

Polaris

Pros:

Multi-vendor-backed Iceberg REST catalog, pluggable RBAC/OPA/Ranger, ASF top-level governance.

Cons:

Needs an external relational backend (Postgres) for persistence.

Swap cost (reversibility):

Medium — Iceberg tables re-register against another REST catalog without rewriting data files; the RBAC/OPA policy has to be re-authored.

Vector

Pros:

Rust-native, fast, declarative GitOps config, built-in VRL test harness.

Cons:

Read-only observability API (no config-push/control API; #24020).

Swap cost (reversibility):

Medium — VRL transforms are Vector-specific, so swapping the ingest engine re-authors every transform. No data rewrite; the landed tables are unaffected.

DataFusion

Pros:

Embeddable Rust engine, no JVM, zero-copy Arrow-native; reads Polaris-Iceberg via iceberg-rust.

Cons:

One narrow edge: additive List schema evolution across mixed-version Parquet (#20835).

Swap cost (reversibility):

Low — reads the shared Iceberg tables through the catalog, so swapping engines repoints clients with no data rewrite.

OCSF

Pros:

Vendor-backed cybersecurity schema (AWS, Splunk, CrowdStrike); nested, no flattening required.

Cons:

List-of-struct columns (e.g. observables[]) stress some engines (see DataFusion #20835).

Swap cost (reversibility):

High — the schema is the contract: moving off OCSF re-maps every ingest transform and re-lands historical data in the new shape.

Provenance:

☒ H-OCSF-CONTEXT-COLLAPSE-01

— verified in MASTER-HYPOTHESIS-TRACKER.md

Swap cost = what swapping that component out costs (a config change vs. a data re-land). Provenance chips tie the pick to a sourced assumption/hypothesis — the public method, not the paid per-vendor Matrix score. Deep dives: [securitydataworks.com/writing](#) · [Matrix](#)

The Capability Matrix — public view

The console gives you the Matrix's public half for free: the component model, the constraint-fit shortlist that **Pick components** computes live from your declared constraints, and what swapping each component back out would cost — the reversibility a risk-averse team reads before it commits to open architecture. The scored half stays paid: the per-criterion 1–5 ratings and the weighted ranking per workload archetype, because that scoring is the SDW deliverable. So you can see here which components clear your constraints and how reversible each pick is, and the scored ranking that says which one fits best is the Capability Matrix at [securitydataworks.com/matrix](#).

DataFusion · Engine · C3

Reversibility — swap cost: Low — reads the shared Iceberg tables through the catalog, so swapping engines repoints clients with no data rewrite.

Polaris · Format + catalog · C1/C2

Reversibility — swap cost: Medium — Iceberg tables re-register against another REST catalog without rewriting data files; the RBAC/OPA policy has to be re-authored.

Vector · Pipeline · C4

Reversibility — swap cost: Medium — VRL transforms are Vector-specific, so swapping the ingest engine re-authors every transform. No data rewrite; the landed tables are unaffected.

Consultant delivery: run with `MOAR_PAID_MODE=1` for the scored view (scores load from the private vault, never this repo).

Thesis Evidence Runner

Re-prove the thesis pillars on demand: each verb runs a `./moar` command against the live stack and reports a dated, Tier-B result (bounded output only — never raw rows).

Evidence verbs (each re-proves a thesis claim against the live stack)

verify

▼

Run Evidence Verbs

Thesis-evidence verbs (./moar)

Select verbs and press Run Evidence Verbs. Each shells out to a `./moar` verb that re-proves a thesis claim against the live stack — answer-equality, the four portability (swap) proofs, and the single-hypothesis checks. With no Docker daemon they report blocked, never a fabricated pass.

Static notebook

This notebook is not connected to a kernel. Any interactive elements will not work.

made with marimo

Intent. Two consultant-mode surfaces plus the evidence runner. The Strategy Vault (OKF) and the Capability Matrix scorecard are paid/consultant IP — in the public default mode the scorecard shows no scores and points to `/matrix`; with `PAID_MODE` on, the consultant sees the per-criterion scored view loaded from the private vault (never the repo). The thesis-evidence verbs re-prove the program's claims live against the stack.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

WalkthroughStartupFlowAnalyze

StrategyConfiguration

Preview source

Zeek conn.log -> Network Activity (4001)

Configuration – raw → OCSF · Network Activity (4001)

Pick a source and watch the deployed mapping turn one **raw event** into an **OCSF record**, field by field. The  rows are the semantic traps a name-based mapper gets wrong — where the schema, not the field name, decides the answer.

Raw event (synthetic; row 1 of 8):

```
{
  "ts": "1767225601.0",
  "uid": "CbenA",
  "id.orig_h": "10.0.1.21",
  "id.orig_p": "49210",
  "id.resp_h": "10.0.0.10",
  "id.resp_p": "443",
  "proto": "tcp",
  "service": "ssl",
  "duration": "2.31",
  "orig_bytes": "520",
  "resp_bytes": "8400",
  "conn_state": "SF",
  "missed_bytes": "0",
  "history": "ShADadFf",
  "orig_pkts": "6",
  "orig_ip_bytes": "840",
  "resp_pkts": "10",
  "resp_ip_bytes": "8760"
}
```

Raw field	Raw value	OCSF field	Value
id.orig_h	10.0.1.21	src_ip	10.0.1.21
id.orig_p	49210	src_port	49210
id.resp_h	10.0.0.10	dst_ip	10.0.0.10
id.resp_p	443	dst_port	443
proto	tcp	protocol_num	6
△ orig_bytes	520	bytes_out	520
△ resp_bytes	8400	bytes_in	8400
orig_pkts	6	packets_out	6
resp_pkts	10	packets_in	10
conn_state	SF	activity_id	2

- string -> IANA number (tcp=6, udp=17, icmp=1)
- △ direction: traffic is from src_endpoint's view and orig_h->src_endpoint, so the originator's SENT bytes are bytes_OUT — a perspective-confused map inverts this (canonical: ocsf/examples Zeek conn_log)
- △ direction: the responder's bytes are bytes_IN (received by src)
- derived, not copied (canonical case table): SF/RSTO->2 Close | S0->4 Fail | REJ->5 Refuse | else->6 Traffic

Produced OCSF record (class_uid 4001, activity_id 2):

```
{
  "class_uid": 4001,
  "activity_id": 2,
  "src_ip": "10.0.1.21",
  "src_port": 49210,
  "dst_ip": "10.0.0.10",
  "dst_port": 443,
  "protocol_num": 6,
  "bytes_in": 8400,
  "bytes_out": 520,
  "packets_in": 10,
  "packets_out": 6,
  "time": 1767225601.0
}
```

Renders the authored OCSF contract from the sample library; the live-router round-trip (the same mapping run by the deployed Tenzir/Vector pipeline) is in Flow › Health.

Schema preview

OCSF 1.8.0 — your data lands in 4 class(es). Every record carries class_uid + category_uid; the key fields per class:

- **Process Activity** (class_uid 1007, category 1) — class_uid=1007, category_uid=1, activity_id (1 Launch), process.name, process.cmd_line, actor.process, actor.user, device.hostname, time
- **Authentication** (class_uid 3002, category 3) — class_uid=3002, category_uid=3, activity_id (1 Logon — the operation), status_id (1 Success / 2 Failure — the outcome), user, src_ip, time
- **Network Activity** (class_uid 4001, category 4) — class_uid=4001, category_uid=4, activity_id, src_ip, dst_ip, dst_port, bytes_in, bytes_out, time
- **API Activity** (class_uid 6003, category 6) — class_uid=6003, category_uid=6, activity_id (CRUD verb), api.operation, actor.user, src_ip, time

SeaweedFS Settings

SeaweedFS Port

8333

S3 Bucket Name

moar-warehouse

Polaris Settings

Polaris Port

8181

Vector Settings

Vector Ingest Port (Syslog TCP)

514

Vector Observability Port

8686

Vector Prometheus Metrics Port

9598

Vector VRL Transform Rule

```
# 1. Parse incoming syslog message as JSON
parsed = parse_json!(.message)

# 2. Map fields to OCSF
Authentication schema (class 3002)
.class_uid = 3002
.category_uid = 3
.activity_id = 1 # logon
.event_time = parsed.timestamp

# 3. Actor/user mappings
```

Save Configuration Spec

Save configuration before deploying.

VRL Testing Console

Validate Vector transform rules before provisioning them in the container.

Sample JSON Record In

```
{
  "message": "log_line",
  "timestamp": "2026-06-18T12:00:00Z",
  "user": "admin",
  "success": true
}
```

Validate VRL Transform

Edit the VRL rule (Configuration tab) and run a validation.

Infrastructure Lifecycle Manager

Spin up or tear down the selected MOAr stack locally in Docker via Pulumi. The data-health gate authorizes the deploy — full breakdown in Flow › Health.

❏ Data-health gate: amber — unproven layers · full breakdown in Flow › Health

Deploy pre-flight

Pre-flight: not ready — 2 blocker(s) to resolve before deploy.

- ❏ Docker daemon — reachable
- ❏ port 8333 — object store (S3) — already in use (another stack? a stale container?)
- ❏ port 8181 — catalog (REST) — already in use (another stack? a stale container?)
- ❏ port 5432 — Postgres (catalog backend) — free
- ❏ object store @ http://localhost:8333 — serving (HTTP 403; bucket 'moar-warehouse' may need init)

Deploy Stack via Pulumi

Tear Down Stack

☐ Override the data-health gate (logged on deploy)

Deployer idle.

Deploy progress

Deploy progress: 0/4 up (4 pending/starting).

- ○ postgres-db (catalog backend) — not created yet
- ○ seaweedfs (object store) — not created yet
- ○ polaris (catalog) — not created yet
- ○ vector (router) — not created yet

Deployment Execution Logs

Static notebook
This notebook is not connected to a kernel. Any interactive elements will not work.

Intent. Configure the stack spec, validate the OCSF transform (VRL) before provisioning, and deploy / tear-down the MOAr stack locally via Pulumi. The deploy is authorized by the data-health gate (shown here as a compact verdict chip; the full breakdown lives in Flow › Health) — you can't deploy onto an incoherent selection without logging an override. Before deploy, the pre-flight checks (Docker, ports, object store), the deploy-progress checklist, and the schema preview show what will come up.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Walkthrough

Startup

Flow

Analyze

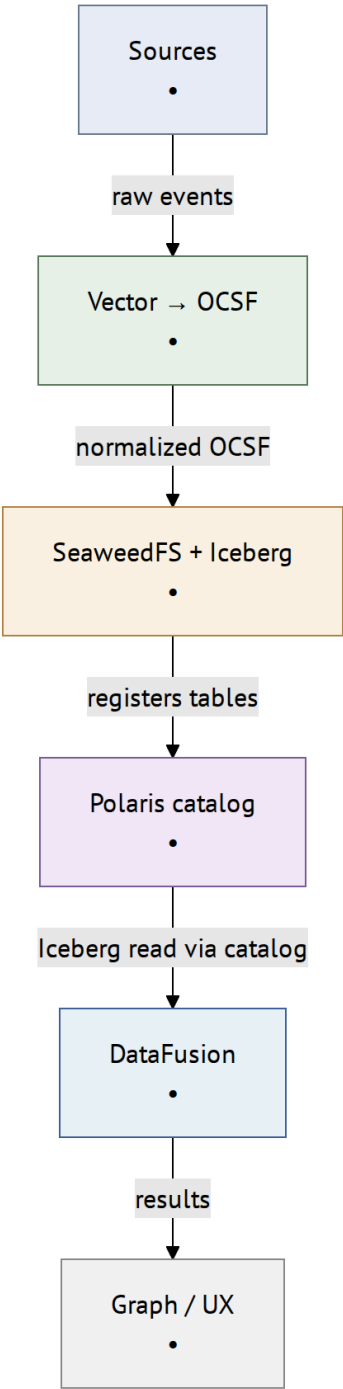
Land

Health

Migrate

Land – pipeline topology

How an event travels through the selected stack – Source(s) → Route (ingest, OCSF-normalize) → Lakehouse (storage + Iceberg) brokered by the Catalog → Engine(s) → Present.



Legend • selected ✓ up ✗ down — unmeasured

No live telemetry attached — all 6 nodes are design-time (selected), not measured. Deploy and wire live status to light the path up.

Static notebook

This notebook is not connected to a kernel. Any interactive elements will not work.

Intent. The pipeline topology — your selected stack drawn as the path an event actually travels: Source → Route (ingest, OCSF-normalize) → Lakehouse (storage + Iceberg) brokered by the Catalog → Engine(s) → Present. A node with no live telemetry shows '—', never a fabricated 'up'; throughput appears only when a real signal carries it.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Walkthrough

Startup

Flow

Analyze

Land

Health

Migrate

Data-Health Gate

Deploy permitted — config integrity holds, but the gate cannot certify GREEN until the unproven layers run.

Config integrity (compatible selection) — pass

Spec persisted — pass

Layer 1 — source health — unmeasured

Layer 2 — stack reachable — pass

Layer 3 — data-quality audit — unmeasured

Layer 4 — cross-tool gap analysis — unmeasured

Cross-engine answer equality — pass

OCSF round-trip (mapping fidelity) — pass

Flow reconciliation (hop counts) — pass

Unproven layers are labeled, never shown as a pass; run the Layer-3 audit (Flow → Health) to turn it green. A green gate is the deploy/inspect authorization, not a slide.

Trust+verify last measured (recorded stack-UP run; decays to stale after a day): answer-equality @ 2026-06-21T22:56:42Z · round-trip @ 2026-06-21T22:57:02Z · flow-reconcile @ 2026-06-21T22:56:54Z

Data Health & Schema Validation (Layers 1 & 3)

Run the data-quality audit (Layer 3, on the selected table) and the source-health audit (Layer 1, across the namespace's sources) together:

Parquet CRC checksum integrity (bit-flip audit)

Schema conformity (OCSF) & NULL constraints

DuckLake tombstone silent-resurrection check (#1215)

S3/SeaweedFS orphan-file audit (manifest match)

Small-file compaction threshold alert (<128MB)

Run Data Health Audits

Capture source baseline (enables completeness)

Layer 3 — data-quality audit

No audit run yet. Deploy the stack, land data, pick a table in the Metadata Inspector, then press Run Data Health Audits. Until then the gate treats Layer 3 as unproven — it never shows an unrun check as a pass.

Layer 1 — source health

No sources audited yet. Deploy the stack, land data, select a namespace in the Metadata Inspector, then press Run Data Health Audits. With no baseline, completeness reads PENDING — you can't measure a drop with nothing to compare to.

Cross-Tool Gap Analysis (Layer 4)

Coverage gaps from an authoritative inventory source to the other tools — exact-match set membership only (entity resolution deferred):

Inventory sources (>=2 tables sharing an identity column)3 selected

Authoritative (primary) sourceauthentication

Identity columnasset_id

Coverage tolerance (max allowed gap)0

Run Cross-Tool Gap

Layer 4 — cross-tool gap analysis

Pick >=2 inventory tables, an authoritative primary source, and the shared identity column, then Run Cross-Tool Gap. Exact-match set membership only — not entity resolution (deferred). Counts only; identities are never rendered.

OCSF Round-trip — Mapping Fidelity (value-level)

Run the deployed router transform over the known raw Okta sample and check each produced OCSF record against the Authentication contract — the value-level meaning a schema-shape pass (Layer 3) can miss. Feeds the data-health gate as a cert-bearing row.

Run OCSF Round-trip

OCSF round-trip (mapping fidelity)

Press Run OCSF Round-trip. The deployed router transform runs over the known raw Okta sample, and each produced OCSF record is checked against the Authentication contract (class 3002, the identity in user, the source in src_ip) — the value-level meaning a schema-shape pass (Layer 3) can miss. With no Docker it reports blocked, never a fabricated pass.

Flow Reconciliation — Hop Counts (emitted → ingested → landed)

Run the real source → route → land pipeline and reconcile per-OCSF-class counts at every hop. A class the pipeline silently loses surfaces as a drop — the coverage hole a reachable Layer 2 can't see. Feeds the data-health gate as a cert-bearing row.

Run Flow Reconciliation

Flow reconciliation (hop counts)

Press Run Flow Reconciliation. The real source → route → land pipeline runs (raw sample → router → Iceberg table), and per-OCSF-class counts are reconciled hop to hop — emitted (source) vs ingested (router output) vs landed (table). A class the pipeline silently drops surfaces here, the coverage hole a reachable Layer 2 can't see. With no Docker it reports blocked, never a fabricated pass.

Live Pipeline Telemetry

Offline

Status

Offline

Events ingested

—

Events delivered

—

Errors

—

No Vector metrics scrape — counters unmeasured (shown as —, never faked). Deploy the stack (Startup → Setup config) and confirm the Prometheus metrics port.

Static notebook

This notebook is not connected to a kernel. Any interactive elements will not work.

Intent. The data-health gate — the console's center of gravity. It refuses to certify the foundation GREEN until each measurable layer actually passes: source health (L1), data quality (L3), cross-tool coverage (L4), plus the cluster-5 deepenings — cross-engine answer-equality, OCSF round-trip mapping fidelity, and flow reconciliation. Unproven layers are labeled, never shown as a pass; a proven layer decays to 'stale' if it isn't re-run. A GREEN gate is the deploy authorization, not a slide.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Walkthrough

Startup

Flow

Analyze

Land

Health

Migrate

Migrate — intent-driven

Pick a migration **intent**; the cockpit expands to focused, verifiable direction — the steps, the swap cost and how to back out, and the live data-health check that proves the move didn't change the answer.

Migration intent

Stand up the lakehouse alongside the SIEM (augment, don't replace) ▼

Migrate — Stand up the lakehouse alongside the SIEM (augment, don't replace)

Run the open lakehouse next to the SIEM and prove parity before you move any read path. Nothing is cut over, so this is the cheapest possible first move — the SIEM stays authoritative while you build confidence that the same questions get the same answers from the lake.

Direction

- Step 1

Tee ingest: fan one copy of telemetry into the SIEM (unchanged) and one into the lakehouse via your selected pipeline, landing as OCSF on SeaweedFS.
- Step 2

Register the landed tables in Polaris and point DataFusion at them.
- Step 3

Pick a handful of existing SIEM detections/searches and re-express them against the lake; run both daily over the same window.
- Step 4

Compare answers side by side until they agree — only then consider moving a read path (the 'replace the SIEM read path' intent).

Swap cost

low — you are adding a stack, not removing one. Backing out is deleting the lake copy; the SIEM is never touched, so there is nothing to undo on the authoritative side.

Reversibility

Fully reversible: tear down the lakehouse copy and you are back to SIEM-only. The SIEM read path is never altered in this intent.

Verify the move

`./moar verify` — cross-engine answer-equality: every running engine must return the identical answer over the same Iceberg table. A new or swapped engine that returns a filtered count short of the others over byte-identical data is a silent-wrong-answer, and this catches it before you trust the read path.

Risks to watch

- Double ingest cost while parallel

You pay to land telemetry twice during the parallel-run window. That is the price of de-risking the cutover; scope it to a subset of sources if the volume bill is a concern, and route-by-value to drop low-value events from the lake copy.
- Parity is the gate, not the calendar

Don't set a cutover date before answers agree. Use `./moar verify` to prove cross-engine answer-equality on the lake first; a date-driven cutover is how a silent mapping gap reaches production.

Static notebook

This notebook is not connected to a kernel. Any interactive elements will not work.

Intent. The intent-driven migration cockpit. Pick one of six migration intents and the panel expands to focused, verifiable direction: the steps, the real swap-cost and how to back out (read off each component's reversibility), and the live data-health check that proves the move didn't change the answer — cross-engine answer-equality for an engine swap, the swap-store/catalog/router checks for those tiers, flow reconciliation for a route change. The open architecture makes being wrong cheap; this makes that concrete.

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

WalkthroughStartupFlow**Analyze**

Detections — hunts over the landed OCSF (aggregate-safe)

Worked detections run over the landed OCSF table. Each result is an **aggregate** — a bounded grouping key plus counts — never a raw event row (real telemetry is a prompt-injection surface). *Worked over the Zeek 4001 sample library (a planted beacon + a high-egress source); the live version runs `run_detections.py` over the landed `ocsf.network_activity` table.*

*

🔍 *C2 beacon — low-byte repeated outbound to a rare destination · T1071 · 1 finding(s)

10.0.1.77 → 203.0.113.66 — connections 3 · avg bytes out 84.0

*

🔍 *Data exfiltration — high outbound volume by source · T1048 · 1 finding(s)

10.0.1.200 — total bytes out 15000 · flows 3

*

🔍 *Credential stuffing / password spray — failed-auth burst from one source · T1110 · 1 finding(s)

203.0.113.99 — failed attempts 6 · distinct users 6

Analyze — log analysis

No table loaded

Load an OCSF table in the Iceberg Metadata Inspector first, then this pane will report aggregate counts over it.

Iceberg Metadata Inspector

List tables and inspect schema/field population from the active REST catalog (counts only — raw rows are never rendered).

🔍 Data-health gate: amber — unproven layers · full breakdown in **Flow** › **Health**

Select Namespace

ocsf

Select Iceberg Table

authentication

Table `ocsf.authentication` (0 rows)

Schema

Q Search...

Visualize

Explore

Export

Field	Type	Required	
str	str	str	
class_uid	int	No	
class_name	string	No	
activity_id	int	No	
user	string	No	
src_ip	string	No	
status	string	No	

6 rows, 3 columns

No selection

Field population

No records yet, or scan failed: When reading information for key `bcsf/authentication/metadata/snap-1499080852949731703-0-fca341b3-7deb-4260-b56e-e216704308d5.avro` in bucket `'warehouse':AWS Error ACCESS_DENIED during HeadObject operation: No response body`

Static notebook

This notebook is not connected to a kernel. Any interactive elements will not work.

made with marimo

Intent. Aggregate-only analysis over the loaded OCSF table — row and per-class/activity counts, top-N sources, field population, time span — plus the Iceberg metadata inspector. It never renders a raw telemetry row: real security data is a prompt-injection and control-char surface, so only counts and low-cardinality keys leave the query, and source values are control-char-stripped and length-bounded.